Option 5: Failure Propagation and Recovery in Multi-Agent Systems

Project description

Multi-agent systems pass intermediate results through messages, delegated tasks, and shared state. An incorrect or malicious result from one agent may influence later decisions by others. This project studies how such failures propagate and how a system can recover without repeating all completed work.

The core question: Can tracking dependencies between agent results support efficient recovery while preserving correct and safe task execution?

For example, a researcher provides a fabricated fact to a planner, and an executor acts on the resulting plan. Once the researcher result is identified as faulty, the system must determine which later outputs remain usable and which need to be recomputed.

Hardware and software environment

Run model inference, agent orchestration, and simulated tools on small AWS CPU instances or your laptops by default. Build a small workflow, such as researcher–planner–executor, with programmatically checkable outcomes.

Phase 1: Fault injection and propagation measurement

Implement and measure a clean baseline workflow. Inject faults at controlled stages: incorrect intermediate results, malicious instructions, timeouts, or stale outputs. Keep accidental faults and adversarial cases separate in the analysis.

Record messages, intermediate artifacts, tool actions, and dependencies. Measure clean task success, failure propagation, unauthorized actions, affected stages, wasted work, and end-to-end latency. Determine whether the workload actually depends on the injected result; an ignored message is not evidence of robust recovery.

Establish full restart as a recovery baseline. Include always verifying intermediate results as a comparison where meaningful, accounting for its extra inference cost.

Phase 2: Selective recovery

Implement one focused recovery mechanism. A suitable default is dependency-aware invalidation: tag intermediate results with their sources, identify downstream results that depend on a faulty output, restore affected simulated state, and rerun only the necessary steps.

Initially supply the identity of the faulty result to every recovery method so that recovery quality is evaluated independently of fault detection. Automated detection is an optional extension and must report false positives and missed faults separately.

To make it more challenging, imagine the case where the environment has changed, and going back to an LLM state in the past doesn't mean other states can also go back in time. For example, your browser state may have changed, you may have overwrote some files, etc. Think about how you can do a proper recovery in those cases. (HINT: one way is to use VM, MicroVM, container to run your agents and take snapshots so you could go back to that state, but this is not the only way)

Compare against full restart and the verification baseline under identical fault schedules and resource budgets. Evaluate faults at different stages and cases involving shared state. Report actions that cannot be undone rather than counting recomputation as complete recovery.

Evaluation and deliverables

Measure recovered task correctness, unauthorized actions, number of recomputed stages, retained valid work, recovery latency, inference cost, and tracking overhead. Check that selectively reused results really remain valid.

Submit the workflow, injection harness, dependency/recovery mechanism, traces, and an analysis of where selective recovery succeeds or fails.

Detailed assessment criteria

Strong submissions distinguish detection from recovery, demonstrate meaningful propagation paths, handle shared-state dependencies, and explain the tradeoff between recovery precision and bookkeeping overhead. No general guarantee against arbitrary malicious agents is expected.

Starting resources

- TAMAS: Threats and Attacks in Multi-Agent Systems
- A2ASecBench